Lesson notes · for you

Schemas and formats

Data engineering · Level 4 · Lesson 1

Time
17 min

Materials
Your phone or a computer, for the Python steps; Optional: a computer with a spreadsheet app and a zip tool

Learning goals

- Choose a schema: normalised for apps, a star for reports.
- Compare row and column formats and compression.
- Parse and type a CSV, counting rows that break the schema.

Activity: Size it on paper

1. Write (4 min): Write three attendance records as CSV lines and as JSON objects, and count the characters of each.
2. Scale (4 min): Multiply up to 25,200 records a school and 20,000 schools. How many megabytes is each format?
3. Star (6 min): Sketch a fact table and three dimensions for district attendance, with their keys.
4. Decide (3 min): Choose a format for sending and one for storing, and say why.

Use made-up records. The point is the ratio between formats, not the exact bytes.

Discussion

- Why does a star schema keep names out of the fact table?
- When would you still choose JSON?
- What should happen to a row that breaks the schema?

Check yourself

- Name one table in a star schema for school attendance, and whether it's a fact or a dimension.

Try it this week
Save the same small table as CSV and as JSON, and compare the file sizes. Then compress both. Which wins, and why?

Worksheet

Schemas and formats

Data engineering · Level 4 · Lesson 1

Name _____ Class _____ Date _____

1 **A report needs the average attendance per class over a year. Which format reads the least data?**

- ◯ One uncompressed CSV file per day
- ◯ A column format: it reads only the columns it needs
- ◯ JSON, because it's readable

2 **Match each piece to its job.**

Normalised schema	• • • •	Reports over millions of rows
Star schema	• • • •	An app that updates data all day
Fact table	• • • •	Each school's name and region
Dimension table	• • • •	One row per pupil per school day

3 **One attendance record, as JSON and as a CSV line. What does this program print?**

```
import json
row = {"date": "2026-03-02",
       "class": "5A", "pupil": 17,
       "present": 1}
line = "2026-03-02,5A,17,1"
print(len(json.dumps(row)), len(line))
```

◯ 18 64 ◯ 18 18 ◯ 64 18

4 **attendance.csv must match the schema: pupil is a whole number, and present is 0 or 1. valid(row) only checks the pupil so far. Add the check for present, so the program prints 6 good rows and 4 bad.**

```
import csv
def valid(row):
    if not row["pupil"].isdigit():
        return False
    return True
counts = {True: 0, False: 0}
with open("attendance.csv") as f:
    for row in csv.DictReader(f):
        counts[valid(row)] += 1
print(counts[True], counts[False])
```

Write your program:

5 **Put the lines in order to create the star schema's fact table, with its columns in this order: date_id, school_id, pupil_id, present.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

- [ ] `school_id INTEGER REFERENCES schools(id),`
- [ ] `present INTEGER`
- [ ] `pupil_id INTEGER REFERENCES pupils(id),`
- [ ] `);`
- [ ] `school_name TEXT,`
- [ ] `CREATE TABLE attendance_fact (`
- [ ] `date_id INTEGER REFERENCES dates(id),`

✎ **Think of data you send or receive often (photos, messages, school results). What format is it in, and what does that cost on a slow or paid connection?**

Check your answers

Schemas and formats

Data engineering · Level 4 · Lesson 1

1 **A column format: it reads only the columns it needs**

The query needs only the class and the status.

2 **Normalised schema — An app that updates data all day**
Star schema — Reports over millions of rows
Fact table — One row per pupil per school day
Dimension table — Each school's name and region

Normalise for writing, use a star for reading.

3 **64 18**

The JSON record spells out every column's name with quotes, colons and spaces: 64 characters.

4

```
import csv
def valid(row):
    if not row["pupil"].isdigit():
        return False
    if row["present"] not in ("0", "1"):
        return False
    return True
counts = {True: 0, False: 0}
with open("attendance.csv") as f:
    for row in csv.DictReader(f):
        counts[valid(row)] += 1
print(counts[True], counts[False])
```

Each rule is one check that returns False.

5 **Top to bottom: 3, 5, 4, 6, X, 1, 2**

```
CREATE TABLE attendance_fact (
    date_id INTEGER REFERENCES dates(id),
    school_id INTEGER REFERENCES schools(id),
    pupil_id INTEGER REFERENCES pupils(id),
    present INTEGER
);
```

The fact table holds keys to its dimensions and one small measure per event.